Data-Driven Fleet Operations: Demand Analytics and Smart Charging for Urban Ride-Hailing in Chicago

Advanced Analytics and Applications

Team 06

Authors:
Louis Schreier (7381065)
Daniel Schumacher (7409287)
Timo Ercole (7408459)
Contact: lschreie@smail.uni-koeln.de

Supervisor: Univ.-Prof. Dr. Wolfgang Ketter
Co-Supervisor: Janik Muires

Chair of Information Systems for Sustainable Society
Faculty of Management, Economics and Social Sciences
University of Cologne

2026-07-26

Table of contents

List of Figures

1 Introduction

1.1 Business Context

A German automotive manufacturer is launching a ride-hailing platform for US customers, operated entirely by an electric vehicle fleet. It's entering a mature market with no operational track record, so it needs data-driven guidance on how to run an electric taxi fleet in a major city.

Bane & Wayne Partners (BWP) was engaged to provide that guidance. This report presents the work of BWP's Data Science Team, which uses historical Chicago ride-hailing data to characterize demand, forecast it, and design a cost-efficient charging strategy.

1.2 Business Goal and Data Mining Goal

The goal is actionable intelligence for launching and running the fleet. The analysis answers three questions:

1. **Where and when is demand concentrated?** This drives fleet positioning and driver scheduling, and tells the client where to enter the market first.
2. **How accurately can future demand be forecast?** Better predictions cut idle time and energy waste, and raise driver earnings.
3. **How should charging be managed to minimize cost?** Charging timed to electricity prices and predicted demand can cut operating cost sharply.

The report is organized into five sections. Section 2 describes the three data sources: the taxi trips, hourly weather, and points of interest. Section 3 describes the cleaning methodology applied that cleans the raw trips and discretizes them onto Uber H3 hexagons to build a forecasting grid. Section 4 uses descriptive analytics to show when and where demand occurs and how trips flow between areas. Section 5 describes the forecasting models used to predict demand per hexagon and time bucket with a Support Vector Machine and a feedforward neural network, then develops a reinforcement learning agent for smart-charging policies under stochastic demand and dynamic pricing. Section 6 summarizes the findings and gives recommendations for the fleet operator.

2 Data Description

The analysis draws on three data sources that together capture demand, its environmental context, and the surrounding urban structure: Chicago taxi trip records, hourly weather observations, and OpenStreetMap points of interest. Full data dictionaries are provided in Section A.

2.1 Taxi Trip Data

The core dataset comes from the Chicago Data Portal and was downloaded on 10 May 2026. It records every taxi trip taken between January 2024 and the beginning of May 2026, amounting to 14,985,679 trips described across 21 columns. Each row corresponds to one journey, and the columns include: identifiers for the trip and the vehicle, start and end timestamps, trip metrics such as duration and distance, spatial fields locating the pickup and dropoff at both census-tract and community-area level, and financial fields covering the fare, tips, total charge, payment method, and operating company. A full breakdown of the fields is given in Section A.

Three characteristics of how the portal publishes this data shape what can be done with it later on. Driver and trip identifiers are released as unique anonymized identifiers, which means the records carry no personally identifiable information while still allowing individual vehicles and trips to be followed across rows. All timestamps are rounded to the nearest 15 minutes for privacy reasons, so the temporal precision is limited, although this leaves hourly and coarser aggregations unaffected. Finally, locations are reported only at the level of census tracts rather than as exact coordinates. With roughly 800 tracts covering the city the spatial resolution is coarser than GPS, but each tract's centroid still provides a usable coordinate for the spatial joins that follow.

2.2 Weather Data

Hourly weather observations come from the Open-Meteo Historical Archive API (Open-Meteo, 2024) and span the same period as the trip data. Chicago covers close to 600 km² and shows genuine microclimatic differences between the lakefront, the dense urban core, and the far northwest, so a single measurement point would not represent conditions across the city well. The weather is therefore sampled at three locations spread across Chicago, which yields 61,335 hourly observations, roughly 20,445 per location. Each observation is timestamped and located, and carries nine meteorological variables that broadly describe temperature, precipitation and snow, wind, and cloud cover.

2.3 Point-of-Interest Data

To capture the urban structure that draws trips, the analysis uses points of interest from OpenStreetMap, retrieved through the `osmnx` library (Boeing, 2017). The query draws on four broad OpenStreetMap tag groups, amenities, shops, tourism, and leisure, which between them describe fixed features of the city such as restaurants, transit stops, hospitals, and shops, and amount to 9,124 locations in total. Each point of interest is essentially a coordinate paired with a type tag. Unlike the trip and weather data, points of interest are static: they do not change over the study period and are never treated as demand themselves, but rather as an indication of how strongly a given area attracts people.

3 Data Preparation

3.1 Cleaning the Taxi Trip Data

Public portal data requires careful validation before it can be relied upon. Rather than imposing a fixed set of rules from the outset, we worked through the dataset issue by issue, diagnosing the cause behind each problem before settling on a targeted fix. This covered missing fields and their imputation, "ghost trips" recordings with zero duration and distance, zero-distance meter cancellations, and trips whose implied speeds or fares were physically implausible; the complete rule set, with per-rule counts, is reported in Section B. Taken together, these steps remove just over 10 % of the raw records without distorting the underlying geography, as the per-neighborhood pickup counts before and after cleaning correlate at 1.0000.

One observation deserves particular attention because it bears directly on the revenue analysis. In 48.0 % of rows the billed total exceeds the sum of its individual components, almost always by exactly \$0.50 and overwhelmingly on card and mobile payments, a pattern that points to an unitemized card surcharge. We therefore apply no correction and treat `trip_total` as the authoritative revenue figure throughout.

Not every following analysis step calls for the same degree of rigor. Since partially faulty trips (e.g., with an unusual speed or fare) still represents a genuine demand event, we distinguish two cleaning levels: **demand forecasting** uses a minimal level that retains 99.99 % of rows, whereas **trip-level analyses** such as fare and duration rely on the full level, which keeps 89.99 %.

The weather and POI data required comparatively light preparation. The three weather zones were obtained by running K-means clustering over all trip pickup coordinates, which placed the zone centroids where demand is densest, namely the downtown core, O'Hare, and the South Side. Beyond this, we dropped a redundant weather condition code and confirmed that the apparent gaps at the spring clock change were correct rather than missing values. The points of interest posed a different problem, as the raw OpenStreetMap tags were too fine-grained and overlapping to be used directly. We therefore collapsed them into seven demand-relevant categories ranging from food and nightlife down to transport, and reduced each geometry to a single coordinate. Aggregated per hexagon as category counts, these then serve as a proxy for how strongly each cell attracts trips (details in Section B).

3.2 From Trips to a Forecasting Grid

The forecasting models do not operate on individual trips but on an aggregated grid, with one row per *(hexagon, time bucket)* and the trip pickup count as the target. Cells that record no pickups are retained as explicit zero-demand rows, ensuring that the models also learn from quiet periods rather than only from busy ones.

Choosing the grain of this grid is ultimately a business trade-off, since finer cells localize demand more precisely but leave a larger share of the grid empty and correspondingly harder to forecast. We settle on **H3 resolution 6 with 4-hour buckets** as our main choice, which gives 27 hexagons $\times$ 5,107 time buckets = 137,889 rows at a manageable 12.6 % zero-demand share. Moving one step finer would push that share toward 50 % without a commensurate gain in operational precision (the full sparsity comparison is given in Section B), and Section 5 examines in detail how coarser and finer grids affect model accuracy.

By design, every model input is knowable before the bucket begins. These inputs comprise the calendar context (hour, day of week, month, and weekend and US-holiday flags, encoded as sine–cosine pairs so that 23:00 sits next to 00:00), the nine weather variables for the nearest zone, and a per-hexagon indicator that allows each cell to learn its own baseline, giving 51 input features per grid cell in total.

4 Descriptive Analytics

4.1 When Demand Occurs

Taxi demand in Chicago follows predictable rhythms that are tied to work schedules, social activity, and weather, and understanding these rhythms is the natural starting point for any decision about fleet positioning and driver scheduling.

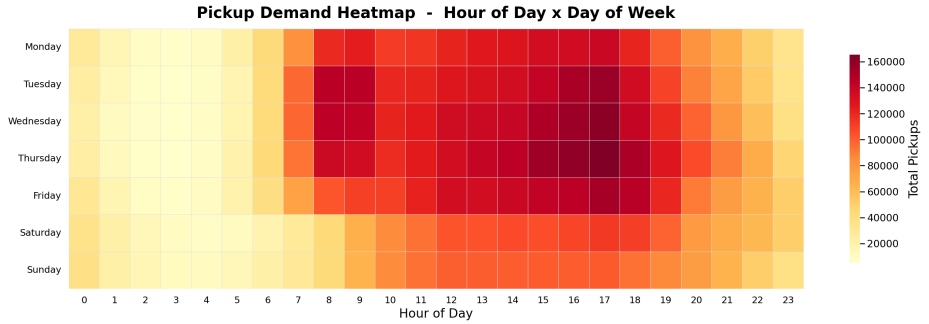

Figure 1: Pickup demand heatmap cross-tabulated by hour of day and day of week.

Weekday demand is distinctly bimodal, with a morning commuter peak around 08:00 followed by a stronger and broader evening peak between 16:00 and 19:00 (probably a reflection of the workday routine), and Tuesday through Thursday carry the highest volumes in the entire dataset. Weekends run on a different clock altogether: demand starts later, builds gradually through the afternoon, and shows none of the commuter spikes, reflecting its essentially leisure-driven character. Across every day of the week, the 00:00–06:00 window remains uniformly quiet, which makes it the natural slot for charging and maintenance in an electric fleet.

Individual trips are, for the most part, short urban hops, with a median duration of around ten minutes, most journeys running under five miles, and fares to match, alongside a long tail of airport and cross-city rides (distributions in Section C). The idle time between a driver's consecutive trips concentrates in the 0–30 minute range, although a mean closer to an hour indicates recurring periods of deliberate waiting or repositioning.

4.2 Where Demand Occurs

To analyze space consistently, we discretize the city with Uber's H3 hexagonal grid, which offers uniform cell areas and clean adjacency where administrative boundaries do not (grid illustration in Section C). The maps below are drawn at resolution 7, while the forecasting models in Section 5 operate one step coarser, at resolution 6.

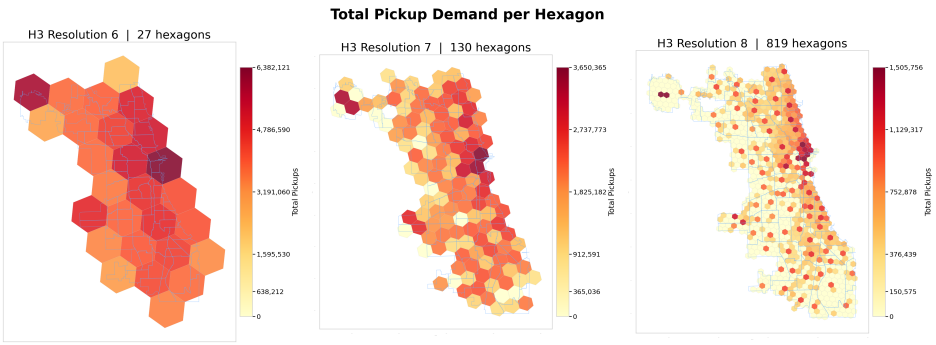

Figure 2: Choropleth maps of pickup demand aggregated at community area level and H3 resolutions 6, 7, and 8.

The Near North Side and the Loop clearly dominate demand, consistent with their role as Chicago's central business and entertainment district, whereas O'Hare forms a structurally distinct cluster in the far northwest. As the resolution increases, the hotspots sharpen into

specific corridors such as the Magnificent Mile, River North, and the area around O'Hare, and neighborhoods that appear moderate at community level turn out to vary considerably within themselves. Community-area aggregation therefore conceals precisely the sub-neighborhood differences that a fleet relies on for its positioning decisions.

The picture shifts over the course of the day, yet the downtown-lakefront corridor remains dominant in every 4-hour window (maps in Section C). Overnight, the downtown core retains relative activity driven by hospitality traffic; early mornings add a secondary hotspot around Midway as the first departures begin; and the evening windows intensify the core sharply while the outskirts stay comparatively quiet.

4.3 Flows Between Areas

Where trips end up matters as much as where they begin. The origin–destination analysis (matrix in Section C) shows the Near North Side and the Loop dominating as both origins and destinations, and O'Hare, despite its distance, sends nearly 500,000 trips toward those two areas alone, which makes the airport-to-downtown corridor one of the most dependable sources of paid mileage in the network. Vehicles that drop passengers in the outer neighborhoods, by contrast, face a structural repositioning problem in getting back to where demand is.

Net flow, defined as pickups minus dropoffs per hexagon, translates this imbalance into something a planner can act on, since persistently positive cells require more vehicles than their dropoffs return, while negative cells steadily accumulate them.

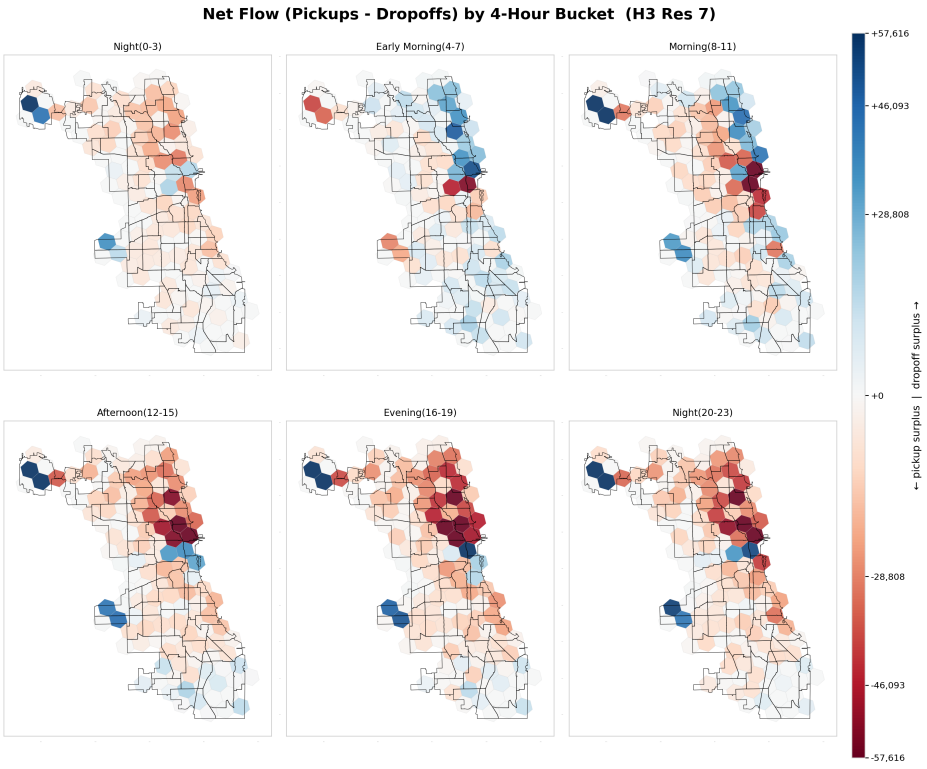

Figure 3: Net flow (pickups minus dropoffs) by H3 hexagon, aggregated across the full observation period.

The downtown core and the O'Hare corridor consistently hand out more trips than they take back, whereas the residential outskirts act as vehicle sinks. For an electric fleet this points

directly at infrastructure, since chargers placed in the negative-flow residential zones would turn otherwise idle accumulation into productive charging time before the vehicles are repositioned to the high-demand corridors.

5 Data Analytics

5.1 Predictive Analytics

5.1.1 Forecasting Problem and Baselines

We set out to predict **taxi demand per hexagon and 4-hour window**, relying solely on information available before the window begins, namely the time of day, day of week, weather forecast, and location. Because a dispatcher scheduling tomorrow's shifts does not yet have tomorrow's actual counts, the models deliberately use no lagged demand. Since trip counts are heavily skewed, with a few downtown cells seeing hundreds of pickups per window while most outer cells see only single digits, we train on log-transformed trip counts and report all metrics back on the trip scale. The grid is split randomly into training, validation, and test data in a 50 / 20 / 30 ratio, and the test set is touched only once, for the final numbers.

To give these results business meaning, we compare them against two naive benchmarks: the *per-cell mean* (the historical average per hexagon and window), scores a MAE of 61.62 trips, while *climatology* (the average per hexagon, hour, and weekday-versus-weekend) scores 23.88. Climatology is the hurdle that really matters, as it is what an operator would already obtain from a simple lookup schedule, and a model therefore justifies its added complexity only by beating it.

5.1.2 Support Vector Regression

We tuned a Support Vector Machine for regression across kernel types and regularization settings on a validation subsample; the search and its intermediate results are in Section D. The best configuration with an RBF kernel (and regularization parameter C = 10), confirms that the link between weather, calendar, and demand is non-linear. Table 1 shows the final test-set results on 41,367 grid cells.

Table 1: SVR test-set performance compared to baselines.

Model	MAE	RMSE	R^2
Per-cell mean	61.62	201.45	0.624
Climatology	23.88	87.86	0.928
SVR (RBF)	**21.78**	**79.74**	**0.941**

The SVR beats both baselines, though its margin over climatology is modest, at an MAE of 21.78 against 23.88, or roughly 9 %. Without access to recent actual counts, most of what the model can learn is the seasonal pattern that climatology already encodes, and the 9 % it adds on top comes from the weather and calendar interactions that a fixed schedule cannot represent. The error is also unevenly distributed: the model is accurate in the many low-demand cells but under-predicts the busy downtown and airport hexagons where forecasts matter most, a known artifact of training on compressed log counts (decomposition in Section D).

Because the granularity of the grid is itself a business decision, we also evaluated the tuned model across coarser and finer grids. Finer cells cut the absolute error per cell, giving 69, 29, and

8.1 trips of MAE at 4-hour windows for resolutions 5, 6, and 7 respectively, simply because each cell holds fewer trips; the explained variance, however, falls off at resolution 7, where roughly half the grid is zero-demand (the full sweep is in Section D). Resolution 6 with 4-hour windows therefore remains the working choice, as moving down to census tracts, roughly 800 units instead of 27, would multiply sparsity without any operational benefit.

5.1.3 Feedforward Neural Network

We trained a feedforward neural network on the same grid and features, so the two models are directly comparable. A grid search over architectures, learning rates, and optimizers (detailed in Section D) selected a network with two hidden layers of 128 and 64 neurons, trained with the Adam optimizer. Table 2 shows the final test-set results.

Table 2: Neural network test-set performance compared to the climatology baseline.

Model	MAE	RMSE	R^2
Climatology	23.88	87.86	0.928
Neural network	**18.90**	**72.42**	**0.951**

The network also beats climatology, and by a considerably wider margin, with an MAE of 18.90 against 23.88, a 21 % improvement. It captures more of the demand structure beyond the weekly rhythm, in particular the way weather, time, and location interact, though a residual bias of -5.34 trips shows that the extreme downtown peaks are still partly missed. Those spikes stem from events the data cannot see, such as concerts and sports fixtures, and would require an event calendar to forecast reliably.

In operational terms, the model produces an on-demand forecast for any combination of location, time, weather, and day type. Queried for an outer-city hexagon on a rainy Sunday in July, for instance, it predicts demand rising from 44.9 trips at 10:00 to a peak of 48.1 at midday before easing to 20.3 by 18:00, amounting to roughly 339 trips across the window and following the leisure-weekend build-up already seen in the descriptive analysis.

5.1.4 Model Comparison

Both models run on the same resolution 6 grid with 4-hour windows, measured against the same climatology baseline:

Table 3: Each model's performance relative to the climatology baseline.

Model	Setting	Climatology MAE	Model MAE	Improvement
SVR (RBF)	H3 res. 6, 4 h windows	23.88	21.78	**Better** (+9 %)
Neural network	H3 res. 6, 4 h windows	23.88	18.90	**Better** (+21 %)

The neural network wins for two structural reasons. It trains on the full dataset, whereas the SVR scales poorly and had to learn from a small subsample, and it learns feature interactions jointly, where the SVR applies one fixed transformation to every input. Both beat the seasonal schedule, but the network improves on it by more than twice as much, and for an operator

positioning vehicles under changing weather or on atypical weekdays that responsiveness is precisely the point, since a seasonal average says nothing about whether tomorrow's rainy Monday will run busier than a dry one.

5.2 Smart Charging

If forecasting tells the operator where demand will be, charging determines what it costs to serve. To study this, we model a single electric taxi that arrives home at 2 p.m. and must leave the garage again at 4 p.m. Rather than charging at a flat rate, an agent sets the charging power every 15 minutes, balancing two competing goals: keeping the recharging cost low and guaranteeing enough energy for the upcoming shift, whose demand is still unknown at the time the charging decisions are made. Because these decisions are sequential and the outcome uncertain, we frame the problem as one of reinforcement learning and train a Deep Q-Network (DQN). The agent observes the battery's state of charge (SoC) and the current time slot, chooses among four power levels (off, 7.3, 14.7, or 22 kW), pays a charging cost that grows steeply with power, and incurs a large penalty if the battery falls short of the realized shift demand. The full environment design, agent setup, and training diagnostics are provided in Section E.

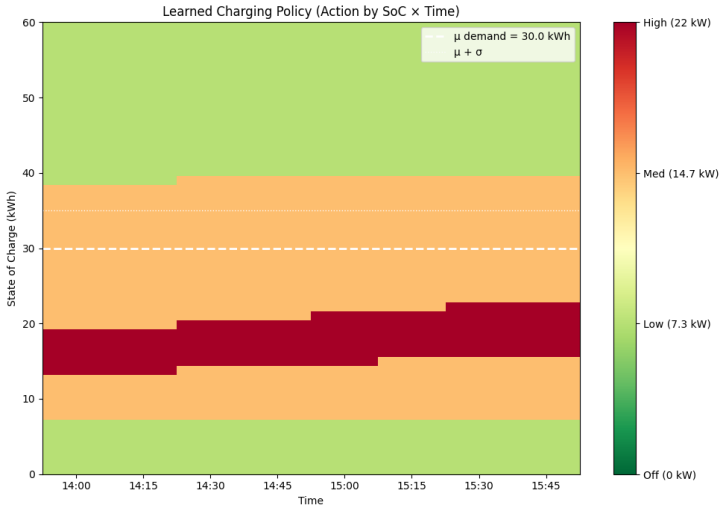

Figure 4: Learned charging policy across SoC and time slot: preferred action (off / low / medium / high) per state.

The learned policy (Figure 4) is simple enough to hand to an operations team, amounting to charging hard while the battery is low, tapering off as a buffer builds, and switching off once the SoC comfortably exceeds expected demand. Because a shortfall is penalized far more heavily than expensive charging, the policy is risk-averse by construction and targets a small surplus above the 30 kWh mean demand rather than the mean itself.

Table 4: Policy comparison over 1,000 evaluation episodes.

Policy	Avg. Reward	Avg. Cost	Avg. Penalty	Avg. Final SoC (kWh)	Shortfall rate
DQN Agent	**-65.10**	64.45	0.65	42.8	**0.4 %**
Always-Max	-86.99	**86.99**	0.00	56.5	0.0 %
Random	-202.91	56.39	146.52	34.4	29.1 %

Evaluated over 1,000 episodes (Table 4), the agent delivers nearly the safety of always

charging at maximum while achieving a far better cost profile. Always-Max never runs short but pays roughly 35 % more in charging cost for a buffer it does not need, ending with an average of 56.5 kWh against a 30 kWh mean demand, and the random policy illustrates the opposite failure mode, since its 29 % shortfall rate would mean roughly one in three shifts starting under-charged. The DQN agent, by contrast, runs short in only 0.4 % of episodes, and this reliability level can itself be tuned through the shortfall penalty.

For the client, the key takeaway is that "charge to full" is not cost-optimal even though it feels safest. Since the state space is only two-dimensional, the learned policy could be deployed as a plain lookup table on a home charging unit. Two caveats nonetheless belong in the client conversation: the agent is trained on a fixed demand distribution and would need retraining if trip volumes shift, and the shortfall penalty is a dial to be set according to the operator's actual tolerance for an under-charged vehicle rather than a fixed technical constant.

6 Conclusions and Recommendations

6.1 What the Analysis Shows

Demand in Chicago is concentrated in space and structured in time. A handful of hexagons account for a disproportionate share of all pickups: the downtown core, the Magnificent Mile, the O'Hare corridor. Most of the city sits close to zero for much of the day. On top of that sits a predictable weekly rhythm: bimodal weekdays with a strong evening peak, later and flatter weekends, and a quiet overnight window common to every day.

Both forecasting models beat the seasonal climatology baseline, the neural network by a clear margin (MAE 18.90 against 23.88) and the SVM more modestly (21.78 against 23.88). The reinforcement learning agent learned a charging policy that reliably met the driver's departure energy needs at a cost well below a naive charge-at-maximum strategy.

6.2 Recommendations for the Fleet Operator

Position vehicles for the corridors, not the map as a whole. Demand lives in a handful of places. That argues for a phased rollout starting in the downtown-to-lakefront corridor and the airport connections, not an even spread across all 77 community areas. O'Hare alone sends close to half a million trips toward the Near North Side and the Loop, so the airport-to-downtown axis is a dependable source of paid mileage from day one.

Use the net-flow signal to plan repositioning. Pickups and dropoffs don't balance locally. Downtown and the airport hand out more trips than they take back, while the residential edges absorb vehicles without generating return demand, and a fleet that ignores this accumulates idle cars in the wrong places. Repositioning logic should be built around the net-flow map, treating the outer sinks as places to hold and charge rather than places to wait for fares.

Forecast for planning, and know what the forecast can't see. The neural network earns its complexity because it responds to weather and atypical days in a way a fixed schedule can't. It's the right tool for shift planning and for questions like expected demand in a given area on a rainy Sunday afternoon. What it doesn't capture are one-off spikes from concerts, fixtures, and conventions. Treat model output as a strong baseline to be overridden by hand on known event days, not as a complete demand picture.

6.3 Private or Public Charging

The charging economics and the demand geography point to a mixed answer. The cost function penalizes high charging power convexly, so charging slowly over a long window is much cheaper than short bursts at full rate. The agent exploited exactly this, spreading energy across the available slots and holding a modest buffer above expected demand. That favors private or depot charging for the bulk of the fleet's energy: it lines up with the quiet overnight window, draws at a low and cheap rate for hours, and can sit in the negative-net-flow residential zones where cars would otherwise idle.

Public fast charging works against this cost structure, because its whole value is delivering energy quickly, which is the expensive corner of the curve. Our recommendation: anchor the fleet on private or depot charging near where vehicles naturally end their shifts, and use public fast charging only as an opportunistic top-up for a car that would otherwise have to leave a high-demand corridor to recharge.

6.4 Limitations

Several caveats bound these findings. Locations are reported at census-tract centroids rather than true GPS, so hotspot detection and flow maps inherit a coarse spatial grain, and fine within-tract patterns are invisible by construction. The forecasting models see no recent demand counts and no event calendar, which is why both under-predict the largest downtown spikes and why the SVR only modestly improves on a seasonal average. The SVR also had to be tuned on a small subsample because it doesn't scale to the full grid. And the charging agent is trained against a stylised environment with a fixed cost coefficient and normally distributed departure demand. Real prices and energy needs vary, so the learned policy is a proof of concept, not a deployable controller.

6.5 Outlook

The most valuable next step is adding lagged demand to the forecasts. The previous period's actual count is usually the strongest single predictor, and its absence is the clearest gap between this setup and a production system. A spatially aware hexagon encoding (distance to center, proximity to transport hubs, neighboring demand) would let the models generalize across cells, and an event calendar would target precisely the spikes they currently miss. On the charging side, real day-ahead electricity prices would let the agent optimize against the tariffs a fleet actually faces. Beyond that, pairing the demand maps with fare-per-mile and idle-time patterns would show where drivers earn most efficiently, and an explicit repositioning model would turn the net-flow findings into concrete vehicle movements.

A Data Dictionaries

A.1 Taxi Trip Data

Table 5 lists the key fields of the trip dataset. Seven temporal features (date, hour, day of week, month, week, weekend flag, holiday flag) are derived at load time, bringing the working total to 28 columns.

Table 5: Core fields of the Chicago taxi trip dataset.

Column	Type	Description
`trip_id`	string	SHA-1 hash; unique journey identifier
`taxi_id`	string	SHA-1 hash; anonymized vehicle identifier
`trip_start_timestamp`	datetime (UTC−6)	Rounded to the nearest 15 minutes
`trip_end_timestamp`	datetime (UTC−6)	Rounded to the nearest 15 minutes
`trip_seconds`	float	Recorded journey duration in seconds
`trip_miles`	float	Recorded journey distance in miles
`pickup_census_tract`	float	Census tract of pickup ($\approx$ 54% missing)
`dropoff_census_tract`	float	Census tract of dropoff ($\approx$ 56% missing)
`pickup_community_area`	float	Community area of pickup (0.05% missing)
`dropoff_community_area`	float	Community area of dropoff (7.2% missing)
`fare`	float	Metered fare in USD
`tips`	float	Tip amount in USD
`trip_total`	float	Total charge billed to the passenger
`payment_type`	string	Credit card, cash, mobile, etc.
`company`	string	Taxi company name
`pickup_centroid_lat/lon`	float	Centroid of the pickup census tract
`dropoff_centroid_lat/lon`	float	Centroid of the dropoff census tract

A.2 Weather Data

The three weather zones are centered on the downtown core (41.897°N, 87.639°W), the O'Hare area (41.980°N, 87.906°W), and the South Side (41.769°N, 87.656°W). Both the elbow method and the silhouette score pointed to $k = 3$. Table 6 lists the hourly variables fetched for each zone.

Table 6: Hourly weather variables fetched from Open-Meteo.

Variable	Unit	Description
`temperature_2m`	°C	Air temperature at 2 m height
`apparent_temperature`	°C	Feels-like temperature (wind chill / heat index)
`precipitation`	mm	Total precipitation
`rain`	mm	Liquid precipitation
`snowfall`	cm	Snowfall accumulation
`snow_depth`	cm	Snow depth on ground
`windspeed_10m`	km/h	Wind speed at 10 m
`windgusts_10m`	km/h	Wind gust speed at 10 m
`cloud_cover`	%	Fractional cloud cover

A.3 Point-of-Interest Data

The OSM query covers four tag categories (`amenity`, `shop`, `tourism`, `leisure`). Raw tags are collapsed into seven demand-relevant categories via a lookup table, grouping tags with similar expected demand timing and dropping types too niche to matter (Table 7).

Table 7: POI counts by functional category (OpenStreetMap, Chicago).

Category	Count	Examples
Food & Nightlife	5,652	Restaurants, bars, cafés, fast food
Education	1,220	Schools, universities, libraries
Shopping	1,085	Supermarkets, clothing, electronics
Healthcare	469	Hospitals, clinics, pharmacies
Entertainment	350	Theaters, cinemas, museums
Accommodation	231	Hotels, hostels
Transport	134	Bus stops, subway entrances, ferry terminals

B Data Preparation Detail

B.1 Missing Values

Different fields are missing for different reasons, so each gets its own treatment (Table 8).

Table 8: Missing-value handling by field.

Field	Missing	Cause	Treatment
Timestamps	< 0.01 %	Fall in the autumn DST fold, where one hour repeats and the occurrence is ambiguous	Remove
Drop-off community area	7.2 %	Drop-off outside the 77 official community areas (suburban)	Recover via spatial join where inside the city; retain suburban trips unassigned

Field	Missing	Cause	Treatment
Trip duration	small	Not reported but derivable	Impute from end − start timestamp
Trip distance	small	Not reported	Estimate with great-circle distance (understates road distance)
Vehicle ID	8 records	Unattributable to any taxi	Remove

B.2 Consistency Checks

Some records are present but aren't real journeys. **Ghost trips** have zero seconds, identical start and end time, and identical start and end location, which looks like a meter switched on and off. All four conditions must hold, so genuine short trips survive; we remove the **152,967** records that qualify (~1 %). A further **1,013,003 trips** (6.8 %) log zero distance despite positive duration, with a median of 73 seconds and a quarter under 7 seconds. These are overwhelmingly meter cancellations or GPS failures rather than real journeys, and all are excluded. We also confirmed there are no end-before-start inversions and no duplicate trip IDs.

B.3 Outlier Removal

After dropping non-journeys, we remove physically implausible values using two derived metrics, average speed and fare per mile (Table 9).

Table 9: Outlier removal criteria applied to the taxi trip dataset; the union of all rules drops 269,407 records (1.96 %).

Criterion	Records flagged	Share of dataset
Non-positive fare	5,252	0.04 %
Distance exceeding 100 miles	1,011	0.01 %
Duration exceeding 3 hours	12,651	0.09 %
Average speed above 70 mph	6,600	0.05 %
Average speed below 2 mph	155,659	1.13 %
Fare below $2 per mile	28,567	0.21 %
Fare above $100 per mile	115,730	0.84 %

B.4 Cleaning Levels

Table 10: Rows retained under each cleaning level.

Level	What is removed	Rows retained
Demand (minimal)	DST-ambiguous timestamps	14,984,551 (99.99 %)
Trip (full)	Above, plus missing vehicle ID, ghost trips, zero-distance trips, and speed/fare outliers	13,485,728 (89.99 %)

B.5 Weather and POI Preparation

Condition codes. The raw weather data carries a WMO condition code (0–99). Treating it as a number implies a false ordering (code 95 isn't "95× worse" than code 1), and only three conditions actually appear: clear, rain, snow. Since rain and snow already have their own numeric columns, the code adds nothing and is dropped.

Clock-change gaps. Each zone shows a one-hour gap on spring-forward days (2024-03-10, 2025-03-09, 2026-03-08). The 02:00 hour doesn't exist when clocks jump 01:00 → 03:00, so the gap is correct and no imputation is needed.

POI geometry. The OSM query returns points, polygons, and lines. All non-point features are reduced to their centroid, so every POI is a single coordinate. Each is mapped to one of seven functional categories via a lookup table, with unmapped types discarded; the final 9,141 POIs are assigned to the hexagon containing their coordinate.

B.6 Granularity and Sparsity

Grain sets a trade-off: finer units capture local variation but leave more cells at or near zero, which is harder to forecast. Table 11 shows the effect.

Table 11: Zero-demand share across spatial and temporal granularities.

Spatial unit	Time bucket	Grid cells	Zero-demand share
H3 resolution 5 (large hexagons)	1 hour	163,400	5.8 %
H3 resolution 6	1 hour	551,475	26.8 %
H3 resolution 6	4 hours	137,889	12.6 %
H3 resolution 7	4 hours	602,626	49.2 %

C Additional Descriptive Figures

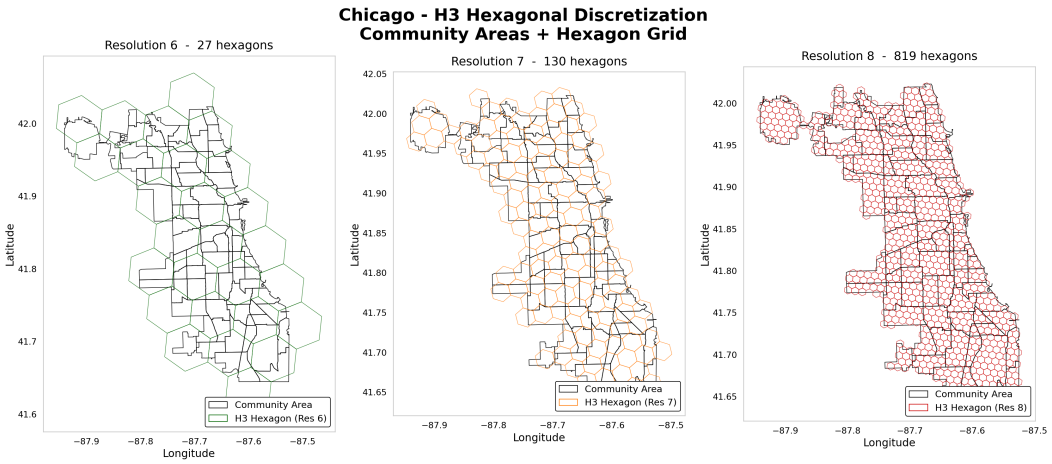

Figure 5: H3 hexagonal grid overlaid on Chicago community areas at resolutions 6, 7, and 8.

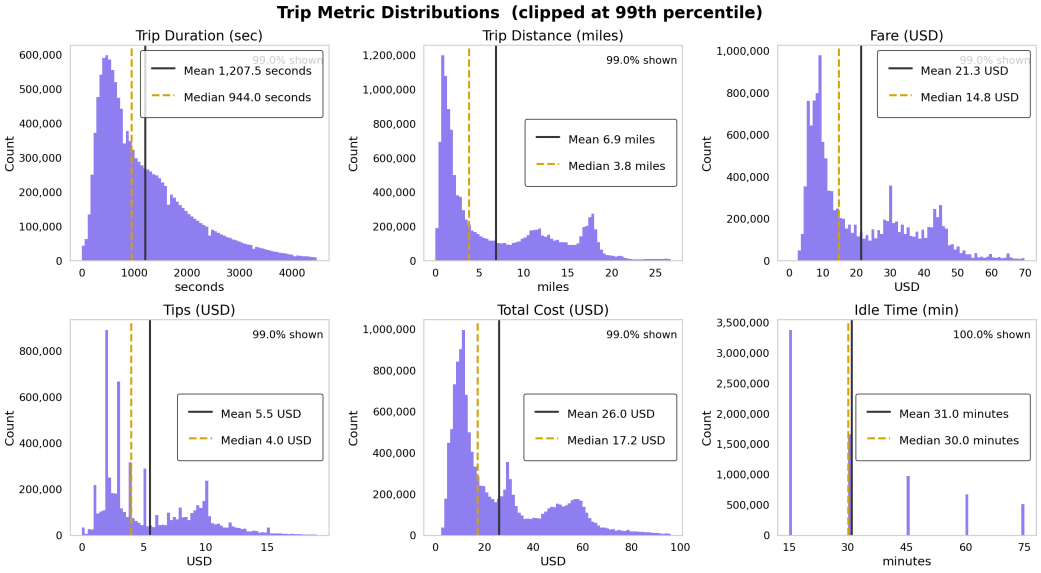

Figure 6: Distribution of trip duration, distance, fare, tips, total cost, and inter-trip idle time across the full dataset. Distributions are clipped at the 99th percentile so airport flat rates and exceptional idle gaps don't distort the axes.

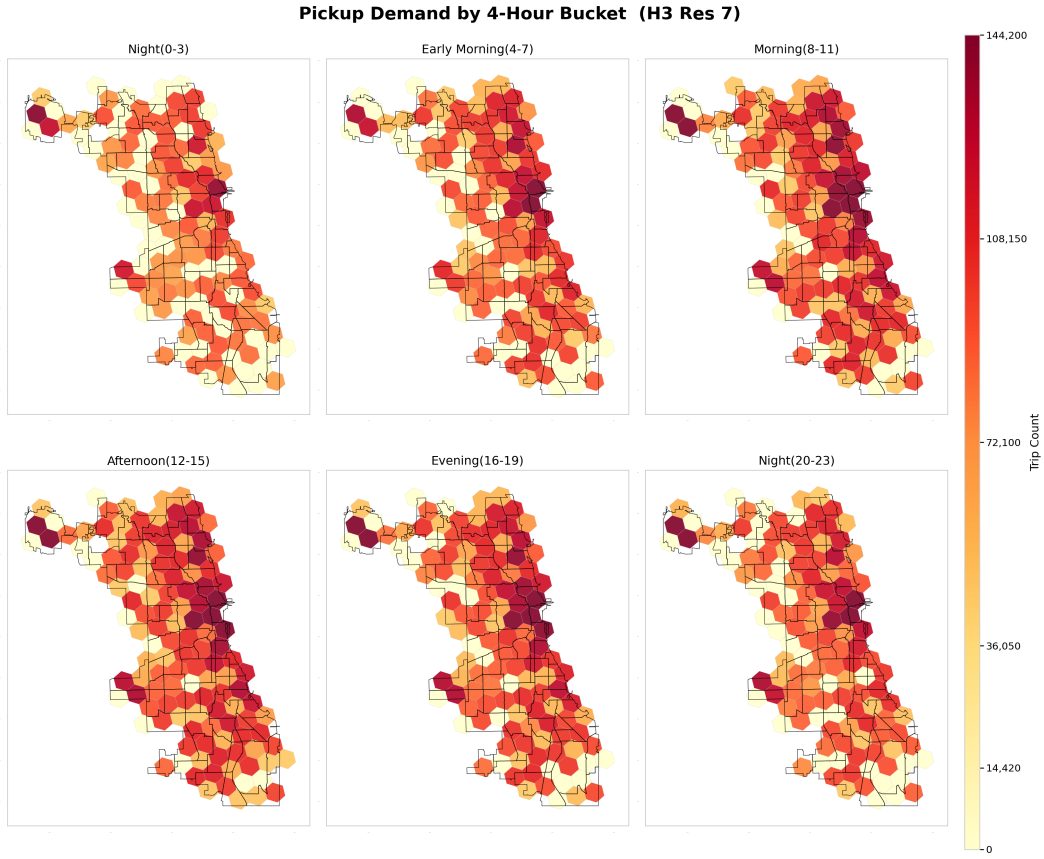

Figure 7: Pickup demand by 4-hour time bucket, covering the full 24-hour cycle.

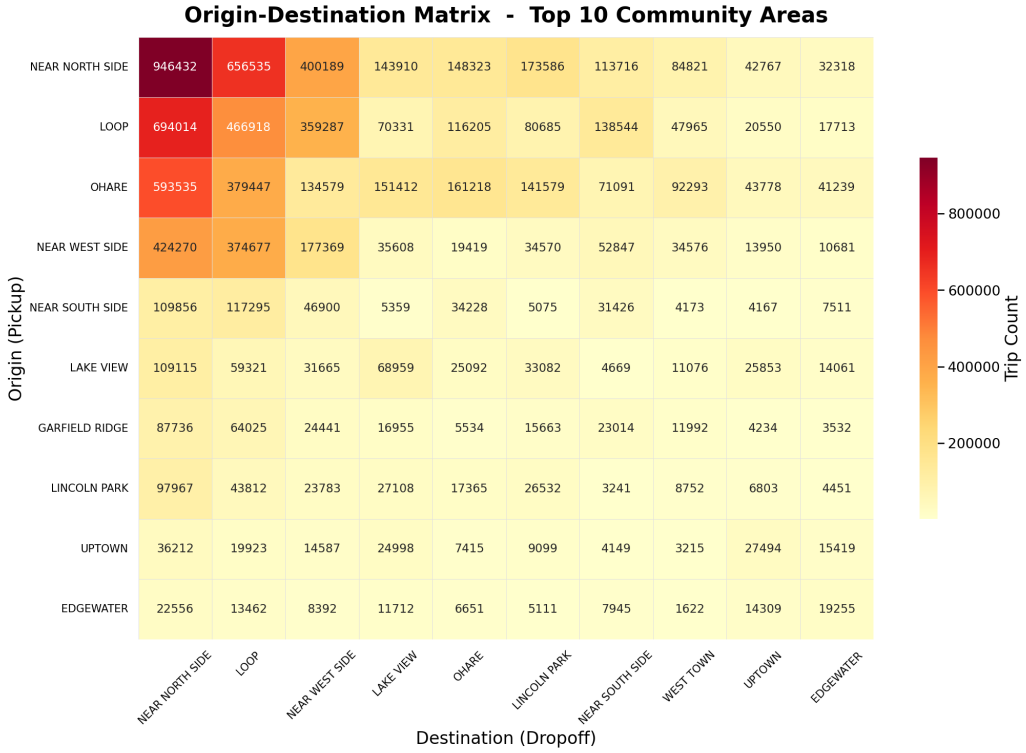

Figure 8: Origin–destination matrix for the top 10 community areas by total trip volume.

D Model Tuning and Diagnostics

D.1 SVR Kernel and Hyperparameter Search

A Support Vector Machine for regression (SVR) fits a function that stays within a tolerance band around the observed values while remaining as smooth as possible. The kernel sets the shape of that function: linear allows only straight-line relationships, polynomial curved ones, and the Radial Basis Function (RBF) highly flexible non-linear relationships. We start simple and add complexity, so that any gain can be attributed to the added flexibility rather than assumed.

Three settings are tuned alongside the kernel: C, how closely the model fits the training data; epsilon, the width of the no-penalty tolerance band; and gamma, how far each observation's influence reaches (RBF and polynomial only). SVR scales poorly with training set size, so tuning ran on a random subsample of 8,000 observations, with the best configuration refit on 30,000 rows for final evaluation.

Table 12: Best SVR configuration per kernel type, scored on the validation set.

Kernel type	Best regularization	Validation MAE
Linear	C = 0.1	36.96 trips
Polynomial	C = 10.0	27.44 trips
RBF	C = 10.0	**23.70 trips**

Moving from linear (MAE 36.96) to RBF (MAE 23.70) shows the value of flexibility: the

link between weather, calendar and demand is non-linear, and the RBF kernel captures that curvature. The RBF kernel with C = 10 and epsilon = 0.1 goes into final evaluation.

D.2 SVR Error Decomposition

Table 13: SVR test-set error decomposed by demand level.

Demand level (trips per window)	MAE	Observations
0–1	0.72	8,373
1–5	1.69	7,417
5–20	3.57	10,232
20–100	11.86	8,406
100+	107.54	6,939

D.3 SVR Performance Across Resolutions

Table 14: SVR performance across spatial and temporal resolutions.

Hexagon size	Time window	MAE	R^2
Large (resolution 5, 8 cells)	1 hour	23.07	0.852
Large (resolution 5, 8 cells)	4 hours	69.32	0.933
Large (resolution 5, 8 cells)	8 hours	106.99	0.943
Medium (resolution 6, 27 cells)	1 hour	7.65	0.884
Medium (resolution 6, 27 cells)	4 hours	28.70	0.899
Medium (resolution 6, 27 cells)	8 hours	42.76	0.944
Fine (resolution 7, 118 cells)	1 hour	2.91	0.754
Fine (resolution 7, 118 cells)	4 hours	8.10	0.820
Fine (resolution 7, 118 cells)	8 hours	11.72	0.887

Finer spatial units and shorter windows lower the absolute error because each cell holds fewer trips. R^2 moves non-monotonically: resolution 7 is consistently the weakest, since roughly half its grid is zero-demand and the per-cell signal gets noisy, while resolutions 5 and 6 trade the lead depending on window width.

D.4 Neural Network Search and Diagnostics

The grid search covers hidden layer sizes ([64, 64], [128, 64], [256, 256] and [64, 64, 64, 64]), dropout (0 %, 10 %), learning rate (0.1, 0.01, 0.001) and optimizer (SGD, Adam). Each candidate trains for up to 200 iterations, stopping early after 20 iterations without validation improvement. The winning configuration (two hidden layers of 128 and 64 neurons, no dropout, learning rate 0.001, Adam) is retrained from scratch on the combined training and validation data.

Holding that architecture fixed, we compare three activation functions (Table 15). Tanh edges out ReLU, and both are far ahead of sigmoid, which never trains to a usable error level because of gradient saturation: its neurons produce near-constant outputs across much of the input range, which stalls learning in deeper networks. The final model uses ReLU; the gap to

tanh is under two trips of MAE, small enough to fall within run-to-run variation on a single validation split.

Table 15: Validation error by activation function for the best network architecture.

Activation function	Validation MAE
ReLU	17.86 trips
Tanh	**17.50 trips**
Sigmoid	81.18 trips

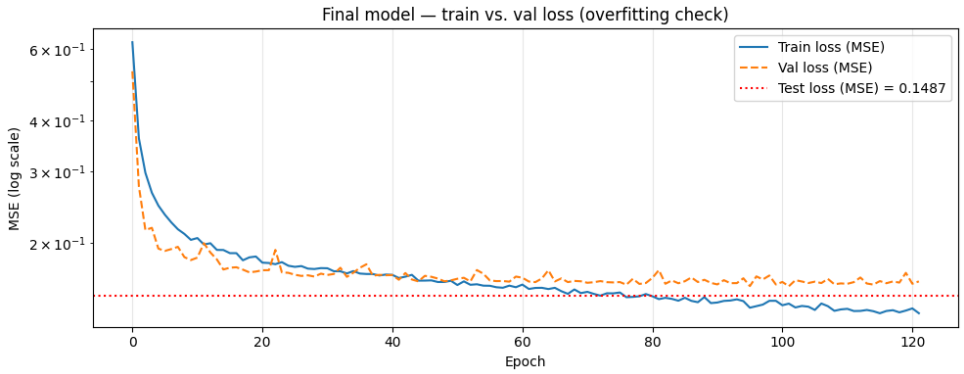

Figure 9: Training and validation loss curves for the final neural network model.

E Smart Charging Environment and Training

E.1 Markov Decision Process Design

State space. The state is a 2-dimensional vector: the current SoC in kWh, and the current 15-minute slot index, normalized to $[0, 1]$. Two components are sufficient because the charging window is fixed (2 hours, 8 slots) and the only quantity that evolves within it is the battery level; calendar or weather features would be redundant, since none of them change within a single 2-hour window. Keeping the state low-dimensional also keeps the learned policy interpretable as a simple heatmap over SoC and time slot (Figure 4), which matters for explaining the agent's behavior to the client.

Action space. Charging power is discretized into four levels (off, low, medium, high: 0, 7.3, 14.7, 22 kW) rather than allowing continuous output. The assignment explicitly permits this simplification, and discrete actions keep DQN applicable without the added complexity of a continuous-control algorithm such as DDPG, which would need substantially more tuning for a 4-way decision that a home charging unit would realistically only support in discrete steps anyway.

Reward function. Every slot incurs a charging cost that grows exponentially in power, $c \cdot e^{p_t/p_{\max}}$, reflecting that fast charging draws a disproportionate premium on real electricity tariffs (modeled here on average ComEd afternoon supply prices) and stresses the battery more than slow charging. At the end of the window, a shortfall between the realized stochastic demand ($\mathcal{N}(30, 5^2)$ kWh) and the final SoC is penalized at a coefficient of 100 per kWh missing, deliberately far above the cost scale of charging itself, so that the agent never learns to gamble on running out of energy to save a few cents. Table 16 summarizes the resulting reward per action, evaluated at the reference cost coefficient of the simulation.

Table 16: Charging cost by action, before any shortfall penalty.

Action	Label	Power (kW)	Energy per slot (kWh)	Cost this slot
0	Off	0.0	0.00	4.00
1	Low	7.3	1.82	5.58
2	Medium	14.7	3.67	7.80
3	High	22.0	5.50	10.87

A note on scaling. An earlier version of the cost function used $c \cdot e^{p_t}$ directly. Because p_t ranges up to 22, this exploded numerically and made the reward signal uninformative for the agent. Normalizing the exponent by $p_{\max}$ keeps the cost in a well-behaved range while preserving the intended convexity: charging faster remains disproportionately expensive, just not to the point of destabilizing training.

E.2 Agent: Deep Q-Network

We use a Deep Q-Network rather than tabular Q-learning or policy-gradient methods, for three reasons tied to this problem's structure. First, the action space is small and discrete (four choices), which is exactly the regime Q-learning-based methods are strongest in; policy-gradient methods (REINFORCE, Actor-Critic) add complexity that buys nothing here. Second, the state space is continuous (SoC is a real-valued kWh amount), so tabular Q-learning would require discretizing it and lose precision at the SoC boundaries near the shortfall threshold. Third, DQN's experience replay lets the agent learn from individual 15-minute transitions rather than waiting for full episodes to complete, which is more sample-efficient than Monte Carlo approaches given the small number of decision points (8) per episode.

The Q-network is a small feedforward network with two hidden layers (12,932 parameters total), trained with a target network and soft updates ($\tau = 0.0005$) for stability, and an ε-greedy exploration schedule decaying from 1.0 to 0.001 over training. Training ran for 5,000 episodes (40,000 transitions).

E.3 Training Behavior

Figure 10 shows the training curves. Reward and shortfall penalty do not decline monotonically: several stretches (around episodes 500 and 1,000) show a spike in average penalty, where the agent temporarily under-charges before correcting. This reflects the exploration-driven trade-off inherent to ε-greedy DQN. The agent must occasionally under-charge to learn the shape of the penalty, since the shortfall penalty is only observed at the end of an episode. By the later episodes, the average penalty converges to zero while average final SoC stabilizes above the 30 kWh demand mean, indicating the agent has learned a conservative buffer against demand uncertainty rather than charging to the mean exactly.

E.4 Evaluation Distributions

E.5 Example Episode

Figure 12 traces one evaluation episode slot by slot: SoC rises from an initial value in the 10–15 kWh range at 2 p.m. to a final SoC of 44.2 kWh at 4 p.m., against a realized demand draw of 24.4 kWh, leaving a comfortable 19.8 kWh surplus at a total charging cost of 66.33. This single trace should not be read as typical (the realized demand here happened to fall below the

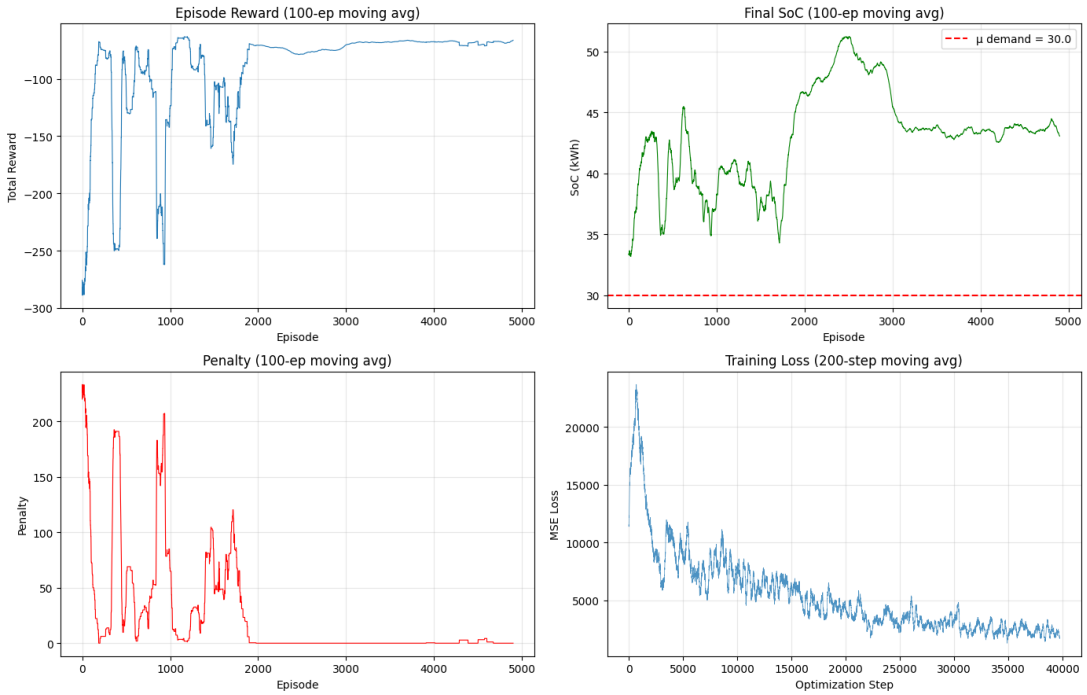

Figure 10: Training curves: episode reward, final SoC, cost and penalty over 5,000 training episodes (100-episode moving average).

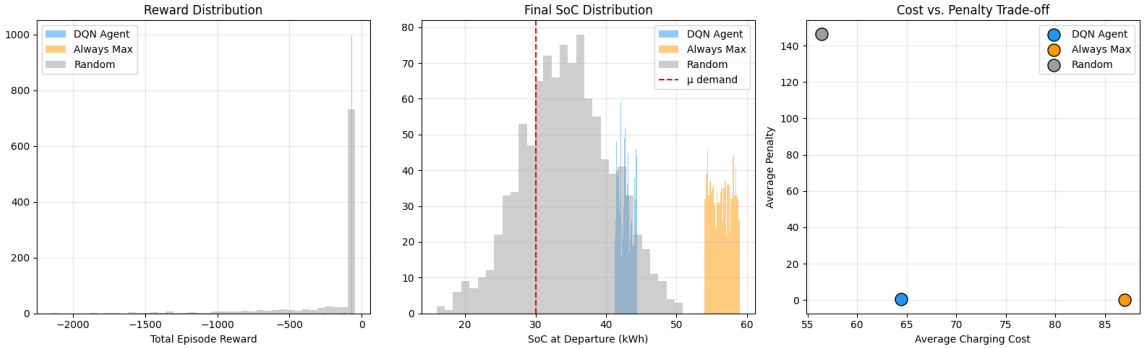

Figure 11: Distributions over 1,000 evaluation episodes: total reward, final SoC, and cost vs. penalty, compared across DQN Agent, Always-Max, and Random.

Figure 12: Example evaluation episode: SoC, charging power, and cost per 15-minute slot from 2 p.m. to 4 p.m.

mean), but it illustrates the slot-by-slot decision pattern: higher power early in the window while SoC is low, tapering off as the buffer builds.

References

Boeing, G. (2017). OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems*, *65*, 126–139. https://doi.org/10.1016/j.compenvurbsys.2017.05.004

Open-Meteo. (2024). Historical weather API [Accessed: 2026-05-10].